csc/cpe 453 Midterm Archive

DRAFT(May 4, 2026 at 13:45)

Name: _____

Section: Any

Rules:

- Do all your own work. Nothing says your neighbor has any better idea what the answer is.
- This exam is closed book. You may use your brain, but you *may not* use any other materials.
-

Suggestions(mostly the obvious):

- When in doubt, state any assumptions you make in solving a problem. **If you think there is a misprint, ask me.**
- **Read the questions carefully.** Be sure to answer all parts.
- Identify your answers *clearly.*
- Watch the time/point tradeoff: Nevermind. There are no points.
- Problems are not necessarily in order of difficulty. They are in the order in which they fit.
- Be sure you have all pages. Pages other than this one are numbered "n of 30".

Encouragement:

- Good Luck!

Problem	Possible	Score
1	•	
2	•	
3	•	
4	•	
5	•	
6	•	
7	•	
8	•	
9	•	
10	•	
11	•	
12	•	
13	•	
14	•	
15	•	
16	•	
17	•	
18	•	
19	•	
20	•	
21	•	
22	•	
23	•	

Problem	Possible	Score
24	•	
25	•	
26	•	
27	•	
28	•	
29	•	
30	•	
31	•	
32	•	
33	•	
34	•	
35	•	
36	•	
37	•	
38	•	
39	•	
40	•	
41	•	
42	•	
43	•	
44	•	
45	•	
46	•	

Problem	Possible	Score
47	•	
48	•	
49	•	
50	•	
51	•	
52	•	
53	•	
54	•	
55	•	
56	•	
57	•	
58	•	
59	•	
60	•	
61	•	
62	•	
63	•	
64	•	
65	•	
66	•	
67	•	
68	•	
Total:	**0**	

1. () (Warm up) How do you spell "Dijkstra"?
2. () (Warm up) At any given time, a process can be in one of three different states. What are the three states and what do they mean?
3. () Under what circumstances would adding timesharing to an operating system be a bad idea? Explain.
4. () What is *multiprogramming?* Under what circumstances would one expect multiprogramming to increase CPU utilization? Why?
5. () What is *multiprogramming?* Under what circumstances would one expect multiprogramming to **decrease** CPU utilization? (i.e. When would multiprogramming lead to *more* wasted CPU cycles?) Why?
6. () For the following, consider a timesharing operating system.
   (a) Can adding timesharing to an operating system ever increase CPU utilization? **Explain.**
   (b) What benefit *does* timesharing offer to the system?
7. () Consider a system where a context switch takes 10ms with a run-to-completion scheduling policy. All jobs are cpu-bound (they never block while running), and the system has a throughput of 20 jobs/sec. What will the CPU utilization be for this system? (As usual, explain...)
8. () Consider a system where a context switch takes 5ms with a run-to-completion scheduling policy. All jobs are cpu-bound (they never block while running), and the system has a throughput of 100 jobs/sec. What will the CPU utilization be for this system? (As usual, **explain**...)
9. () Consider a system with a run-to-completion scheduling policy and a workload that is completely cpu-bound (jobs never block while running). If this system has a throughput of 50 jobs/second and a utilization of 80%, how long does a context switch take on this system?
   (As usual, **explain**...)
10. () Consider the LWP assignment. In class, when we were discussing how to wind up the stack in `lwp_create()` I told you that it didn't matter what value you choose for the initial "saved" base pointer value, but that you would have to convince yourselves that that is true. Why is this true? **Explain everything that happens to it between creation and destruction.** If it matters, you may assume that the thread's function returns rather than calling `lwp_exit()`.
11. () Consider a system with a run-to-completion scheduling policy and a workload that is completely cpu-bound (jobs never block while running). If the system has a throughput of 50 jobs/sec, what is the maximum amount of time that can be spent per context switch if we wish to maintain a utilization of 90%? (As usual, **explain**...)
12. () Consider a system with a run-to-completion scheduling policy and a workload that is completely cpu-bound (jobs never block while running). If the system has a throughput of 100 jobs/sec, what is the maximum amount of time that can be spent per context switch if we wish to maintain a utilization of 50%? (As usual, **explain**...)
13. () Consider a system with a run-to-completion scheduling policy and a workload that is completely cpu-bound (jobs never block while running). If the system has a throughput of 100 jobs/sec, what is the maximum amount of time that can be spent per context switch if we wish to maintain a utilization of 90%? (As usual, **explain**...)

14. () Consider a system with a run-to-completion scheduling policy and a workload that is completely cpu-bound (jobs never block while running). If the system has a throughput of 50 jobs/sec, what is the maximum amount of time that can be spent per context switch if we wish to maintain a utilization of 80%? (As usual, **explain**...)

15. () Consider a system with a run-to-completion scheduling policy and a workload that is completely cpu-bound (jobs never block while running). If the system has a throughput of 100 jobs/sec, what is the maximum amount of time that can be spent per context switch if we wish to maintain a utilization of 70%? (As usual, **explain**...)

16. () Consider a system with a run-to-completion scheduling policy and a workload that is completely cpu-bound (jobs never block while running). If the system has a throughput of 50 jobs/sec, what is the maximum amount of time that can be spent per context switch if we wish to maintain a utilization of 80%? (As usual, **explain**...)

17. () When an operating system's scheduler chooses a user process, exits supervisor mode, and yields the CPU to that process, how does the operating system guarantee that it will regain control of the machine? **Explain.**

18. () We have discussed two major types of schedulers, preemptive and non-preemptive. The functioning of a non-preemptive one is easy to visualize, each process voluntarily cedes control back to the operating system, but a preemptive one is more problematic because the operating system is not running. How does an operating system with a preemptive scheduling policy preempt a running process and regain control of the machine? **Explain.**

19. () What is the fundamental difference between a process and a thread?

20. () What is a *race condition?* What are the symptoms of a race condition?

21. () Semaphore waiting lists are often implemented as queues served in FIFO order. Could they be implemented as stacks? What problems might this cause?

22. () Semaphore waiting lists are often implemented as queues served in FIFO order. When processes are waiting, an UP() operation wakes up the process at the head of the queue. In class I might say that this is logically equivalent to incrementing the semaphore, waking up *all* the waiting processes and letting them each try again to DOWN() the semaphore. That statement was only *mostly* true. What problem is solved by the queues that is not solved by my proposed alternative? **Explain.**

23. () Semaphore waiting lists are often implemented as queues served in FIFO order. Could they safely be implemented by randomly choosing any one of the waiting processes to wake? What problems might this cause? **Explain.**

24. () Busywaiting is often denegrated in the concurrent programming community as a crude and inefficient practice. However, it has its place.

    (a) First, what is busywaiting?

    (b) Under what circumstances would it be appropriate to choose a busywaiting approach?

25. () In class I said that busywaiting solutions to synchronization problems were only possible on systems that were either timeshared or truly parallel. Why is this so?

26. () Is it possible to use a busywaiting solution to synchronize processes on a single-processor system that is multiprogrammed but not timeshared? Why or why not?

27. () After a scintillating lecture on deadlock, two Operating Systems students are arguing in the hall. The first, Harry Hacker, says that to avoid deadlock all processes must acquire resources in the same order and release them in the opposite order. The other, B. Leary Vision, says that it doesn't matter what order they're released. Who is right and why?

28. () Is it possible to have a deadlock involving only a single process? Why (and give an example) or why not?

29. () Ten point deadlock problem to be named later (I hope).

30. () In 1971, Coffman, *et al,* described a set of conditions that are necessary for there to be a deadlock in a system. What are they and what do they mean?.

31. () When discussing synchronization, we spend a lot of time talking about *critical sections.* What's critical about them, and why is it important to synchronize around them?

32. () In every location where children gather there always seems to be one *Favorite Toy (FT)* that is better than any other option and which the children involved do not want to share. Assume we model each child as an independent process and guarantee exclusive access to whichever child is holding the toy. Is deadlock possible in this system? **Why or why not?**

33. () Consider the threading package you implemented as Asgn2 (`liblwp`). Assume you needed to implement a producer/consumer-type application using these threads. Would it be appropriate to use Peterson's Solution to synchronize your threads? **Why or why not?**

34. () In class I said that busywaiting solutions to synchronization problems were only possible on systems that were either timeshared or truly parallel. This is bad news for our LWP package. What simple modification could be made to our busywaiting algorithms that would allow them to to work with LWP? (**Explain why,** of course.)

35. () Give an example of a race condition that could possibly occur when buying airplane tickets for two people to go on a trip together. Be sure to explain what the problem is.

36. () The lightweight processing system implemented as Assignment 2 did not include any functionality analogous to `fork()`. Below, describe how you would implement `lwp_fork()`. You don't have to actually implement it, but be sure to include enough detail that it is clear that you could. For each step be sure to describe: (a) *what* is to be done, (b) *why* it is necessary, and (c) *how* it could be accomplished. If there is something that cannot be done, explain why.

    `Lwp_fork()` takes no arguments and returns nothing. A call to `lwp_fork()` creates a new thread executing at the same place as the original. Their histories are the same, but their futures may be different.

    A quick reminder of the data structures involved in the LWP system:

```
lwp_context lwp_ptable[];       /* the process table. One entry per thread */
int lwp_procs;                  /* the number of thread contexts in lwp_ptable */
int lwp_running;                /* the index of the currently running thread in lwp_ptable */
typedef struct context_st {
    unsigned long pid;          /* lightweight process id (unique for each thread) */
    unsigned long *stackbase;   /* location of the thread's stack (as returned by malloc()) */
    unsigned long stacksize;    /* Size of allocated stack in words (sizeof(unsigned long))*/
    unsigned long *sp;          /* current value of thread's stack pointer */
} lwp_context;
```

    Think carefully, think a little more, *then* write[1].

    Optional extra space for problem 36.

[1] You are under no obligation to fill all the space available. Some people write larger than others.

37. () Given a user-level non-preemptive lightweight process library similar to that which you have already implemented comprising the following functions(just in case, all of `lwp.h` included on p.**??**):

`lwp_create(function,argument,stacksize)`	create a new LWP
`lwp_gettid(void)`	return thread ID of the calling LWP
`lwp_exit(void)`	terminates the calling LWP
`lwp_yield(void)`	yield the CPU to another LWP
`lwp_start(void)`	start the LWP system
`lwp_stop(void)`	stop the LWP system
`lwp_set_scheduler(scheduler)`	install a new scheduling function
`lwp_get_scheduler(void)`	find out what the current scheduler is
`tid2thread(tid)`	map a thread ID to a context

Now, implement semaphores for these lightweight processes **without modifying any of the above functions**. Provide the type definition for the type `Semaphore` and then define the following three functions:

`Semaphore *newsem(int num)`	creates a new semaphore with an initial value of `num`
`void down(Semaphore *s)`	standard meaning of down for a semaphore
`void up(Semaphore *s)`	standard meaning of up for a semaphore

Since you cannot modify the original library it might be useful to be able to keep track of particular threads. For this, you may want to use the `lwp_gettid()` function and a pre-defined queue of thread IDs type, called `Queue`, with the (usual) functions:

`Queue *newqueue(void)`	creates and initializes a new empty queue
`tid_t empty(Queue *q)`	returns true of the queue is empty, false otherwise.
`tid_t head(Queue *q)`	returns the value at the head of the queue
`tid_t dequeue(Queue *q)`	removes the value at the head of the queue
`void enqueue(tid_t val, Queue *q)`	adds the given value to the end of the queue

You may assume that the current scheduling function is a round robin scheduler.

Important: Explain briefly how your solution solves the problem.

Think carefully, *then* write. (This is simpler than it first appears.) Solutions will be graded both for correctness and quality.

Optional extra space for problem 37.

38. () Would a solution to problem 37 still work if the library were either changed to use preemptive scheduling, or moved to a multiprocessor with more than one thread scheduled at a time? (For this question, assume that your solution to problem 37 is correct, whether or not you believe it to be.) **Justify your answer.**

39. () Consider the following program which uses UNIX/MINIX system calls and refers to `/bin/date` which is a program that prints today's date. For the purposes of this question, you may assume that no system errors occur during the execution of the program and that `printf(3)` executes atomically.

```
int main(int argc, char *argv[]){
  recurse(0);
  exit(0);
}

void recurse(int i) {
  int p, status;
  if ( i > 3 )
    execl("/bin/date","date",NULL);
  p = fork();
  if ( !p ) {
    printf("Apple %d\n",i);
    recurse(i+1);
  } else {
    wait(&status);
    printf("Orange %d\n",i);
  }
}
```

Answer the following questions concisely, but justify your answers.

(a) Is the output of this program deterministic or does it depend on the scheduler?

(b) What is the greatest number of processes that will be active at any time during a run of this program? (Do not count the shell.)

(cont.)

(c) What will be the output due this program being run? (State any relevant assumptions you make.)

40. () What will be the output of the following program? Explain.

```
int main(int argc, char *argv[]){
  printf("Hello.\n");
  fork();
  fork();
  fork();
  printf("Goodbye!\n");
  return 0;
}
```

41. () Given the following program:

```
int main(int argc, char *argv[]){
  int n;
  for(n = 1; n < argc; n++ )
    execvp( argv[n], &(argv[n]) );
  return 0;
}
```

Suppose that it is compiled in the current directory as `myprog` and that one gives the command:

```
% myprog myprog ls myprog myprog myprog
```

(a) How many times will `execvp()` be called during the run of this program?

(b) What will be the expected output of the run?

(c) Explain.

42. () A programmer dissatisfied with the behavior of a C library function can redefine it without limiting the capabilities of the program. (That is, there is nothing the program could have done before the redefinition that it could not do afterward.) A system call, however cannot be replaced without limiting the program. Why is this?

43. () In William Golding's book, *The Lord of the Flies,* a group of school-age boys is stranded on an island. In order to organize themselves and maintain order[2] they make a rule about their meetings: *only the boy holding the conch shell is allowed to speak.*

Write some code to model this process using semaphores and the C-like syntax used for semaphore examples in class and in Tanenbaum and Woodhull. Assume each boy is an independent process.

Assume for the purposes of this problem there are N boys arranged in a circle, that N is a constant, and that numbers increment in the clockwise direction. You may also assume that each boy has something to say, and when a boy is finished talking he passes the conch to his left.

Write three functions to manage the meeting:

`start_talking(int i)`	called by boy i when he wants to speak.
`stop_talking(int i)`	called by boy i when he is done speaking.
`init()`	called before anybody tries to speak.

Thus, the `boy()` function would look something like:

```
void boy(int self) {
  for(;;) {
    start_talking(self);
    /* spout nonsense */
    stop_talking(self);
    /* shaddup */
  }
}
```

Explain briefly how your solution models the process of passing the shell and label any shared data.

Write your code below:

Optional extra space for problem 43.

44. () Consider the following situation[3]: A very narrow hurricane has washed out all but one lane of the Lake Pontchartrain Causeway[4]. Given that it is a very large lake, going around is impractical, so it is necessary to come up with a system to keep the bridge open. The conditions:

- Cars arrive at random intervals from either the north or south.
- The remains of the bridge are only one car wide and cars cannot back up. That is, a car that meets another car is stuck forever.
- Whenever a car wants to enter the bridge, it calls the function `enter_bridge(int direction)` with a pre-defined integer constant indicating the direction. This will be either `NORTH` or `SOUTH`. When it wants to leave, it calls `exit_bridge(int dir)`.

[2]At least this is the case initially. Read the book.

[3]Note, this problem is too hard for a midterm, but it's a good problem to consider, so I'm leaving it here.

[4]Lake Pontchartrain is a lake 41 miles long and 24 miles wide north of New Orleans, La. The causeway is the bridge that spans the "short" direction and is one of the two roads out of the city.

Using semaphores and the C-like syntax used for semaphore examples in class and in Tanenbaum and Woodhull, develop a solution to the problem. Implement `enter_bridge()` and `exit_bridge()` and whatever auxilliary data and functions you may need. **Be sure to explain briefly why your solution works.**

For partial credit: produce a solution that allows cars to cross the bridge without risking meeting another car on the way (and getting stuck forever).

For more partial credit: produce a solution that guarantees that no car will have to wait forever to cross.

For full credit: produce a solution that does all of the above and allows multiple cars travelling in the same direction to be on the bridge at a time. (It is 24 miles long, after all.)

Write your code below (and on the next page if necessary):

Optional extra space for problem 44.

45. () In a 4-man bobsled race 4 bobsledders voluntarily climb into a scary-looking contraption and launch themselves down an icy track at insane speeds[5]. In this problem we are concerned with enforcing both fairness and (marginal?) safety at the track.

- The N teams are modeled as independent processes numbered $0 \ldots N-1$.
- No more than one team may be in the track at a time.
- Teams take turns in numerical order, and each team always wants to take its next turn.

Using semaphores, develop a solution to the problem by implementing the following three functions:

`enter_track(int i)`	called by team i when it wants to enter the track.
`exit_track(int i)`	called by team i when it exits the track.
`init()`	called before any team tries to enter the track.

You may make no assumptions about the order in which blocked processes are awakened.

Partial Credit: If you are unable to do this develop a scheme that abandons fairness but only allows one team at a time to be in the track.

Explain briefly how your solution enforces the rules above.

Optional extra space for problem 45.

46. () You have been chosen by the Fire Marshal to enforce the maximum occupancy limits for classrooms in building 14 (lucky you, eh?). Your task: Using semaphores, and the C-like syntax we used in class ("`Semaphore`" to declare a semaphore and "`UP()`" and "`DOWN()`" for the operations), develop a solution to the synchronization problem described above by implementing the following three functions:

`enter_room()`	called by a student who wants to enter the room from outside.
`exit_room()`	called by a student who wants to exit the room and go outside.
`init()`	called at the beginning of the day before any students come to campus.

- All students are modeled as independent processes. There is no limit on the number of students.
- No more than N students may be in the classroom at a time.
- It's a rainy day, and the hallway leading to the classroom is mostly blocked by other students sitting on the floor talking on their phones. It is impossible to attract the attention of a phone-talker, so there only is room for one student to walk through the hall to or from the classroom. No more than one CSC/CPE/SE/EE student may be in the hallway at any given time.

[5] They consider this fun.

- Describe actions with comments. E.g.: *"/* enter hallway */", "/* leave classroom */", etc.*

Explain briefly how your solution solves the problem.

Think carefully, *then* write[6]. Solutions will be graded both for correctness and quality.

Optional extra space for problem 46.

47. () One of the exhibits at the Ronald Reagan Presidential Library is a retired Air Force One (pretty cool unless you happen to have a freaked-out toddler with you). To see the plane, groups of visitors wait in line until there's space on the plane. Then they pose for a picture (with their group alone on the steps, waving in a presidential fashion) and enter the aircraft. No more than 15 groups are allowed on the aircraft at a time (a Boeing 707 isn't as big as it used to be, after all). As soon as one group exits the aircraft, the next group is allowed to start.

    Using semaphores, and the C-like syntax we used in class ("`Semaphore`" to declare a semaphore and "`UP()`" and "`DOWN()`" for the operations), develop a solution to the synchronization problem described above by implementing the following three functions:

`enter_plane(int i)`	called by group i when it wants to enter the plane.
`exit_plane(int i)`	called by group i when it exits the plane.
`init()`	called before any group tries to enter the plane.

    You may assume that the semaphores are implemented with queues for waiting processes. Use comments to describe when various actions take place such as boarding the plane, waving presidentially, etc.

    If you can't do the full solution, implement a version that simply keeps too many groups from being on the plane at once (for less credit, of course).

    Clearly state any assumptions you make, and explain briefly how your solution enforces the rules above.

    Optional extra space for problem 47.

[6] You are under no obligation to fill all the space available. Some people write larger than others.

48. () In most park playgrounds there is one slide that is better than all the rest, down which all the children want to slide. Because children are seriously interested in fairness (and their parents are interested in safety) we will concern ourselves with enforcing both fairness and safety on the slide.

    For fairness' sake, each child will be modeled as an independent process numbered $0 \ldots N-1$. Whenever a child exits the slide, he/she runs back around to take a place at the end of the line. That is, no child will ever give up his/her turn[7]. For safety's sake, no more than one child may be on the slide at a time[8].

    You may assume that all the children $(0 \ldots N-1)$ arrive at the park in numerical order and stay forever[9].

    Using semaphores, develop a solution to the problem by implementing the following three functions:

`start_sliding(int i)`	called by child i when it wants to start sliding.
`finish_sliding(int i)`	called by child i when it gets off the slide.
`init()`	called before any child tries to enter the slide.

    You may make no assumptions about the order in which blocked processes are awakened.

    Partial Credit: If you are unable to do this, develop a scheme that abandons fairness but only allows one child at a time to be on the slide.

    Explain briefly how your solution enforces the rules above.

    Optional extra space for problem 48.

49. () Imagine a busy pizzeria with an indeterminate number of cooks making pizzas as fast as they can.

    - All cooks are independent processes, and there is no limit on the number of them, all doing the following:

```
void cook(int self) { /* self is unique per cook */
  while ( !quitting_time() ) {
    /* make_pizza */
    add_pizza(self);
    /* let cook */
    remove_pizza(self);
  }
}
```

    - No more than N pizzas may be in the oven at a time.
    - Pizza ovens are *way hot* so cooks must use a long-handled pizza peel to move pizzas in or out of the oven.
    - The restaurant manager is cheap. There is only one peel.
    - Describe actions with comments. E.g.: *"/* put pizza in oven */", "/* take peel*/", etc.*

[7] Until dragged home for dinner, which is beyond the scope of this problem.
[8] Grown-ups are such a drag.
[9] Dinner-schminner.

Using semaphores, and the C-like syntax we used in class ("`Semaphore`" to declare a semaphore and "`UP()`" and "`DOWN()`" for the operations), develop a solution to the synchronization problem described above by implementing the following three functions:

`add_pizza(int owner)`	called by a cook who wants to add a pizza to the oven.
`remove_pizza(int owner)`	called by a cook who wants to remove a pizza from the oven.
`init()`	called at the beginning of the day before the restaurant opens.

Explain briefly how your solution solves the problem.

Think carefully, *then* write[10]. Solutions will be graded both for correctness and quality.

Optional extra space for problem 49.

50. () Imagine a clockshop full of dozens of mechanical clocks. Further imagine the the clockmaker is... particular and simply can't stand to hear "tick... tick" or "tock... tock". Our clockmaker does not care which clock tocks after each tick or ticks after each tock, but a harmonious "tick... tock" should be the only sound throughout the day.

- Assume for the purposes of this problem that there are N clocks running concurrently, each of which runs the following function:

```
void clock(int self) {
  while(!wounddown()) {
    tick(self);
    /* whatever clocks do when they're not ticking */
    tock(self);
  }
}
```

- The clockmaker doesn't want any clock held up longer than necessary. That is, you should attempt to maximize concurrency in your solution (after correctness, of course).

Using semaphores, and the C-like syntax we used in class ("`Semaphore`" to declare a semaphore and "`UP()`" and "`DOWN()`" for the operations), develop a solution to the synchronization problem described above by implementing the following three functions:

`void tick(int id)`	Called when a thread wants to tick.
`void tock(int id)`	Called when a thread wants to tock.
`void init()`	Called at the beginning of time before any clocks are created.

Explain briefly how your solution solves the problem.

Think carefully, *then* write[11]. Solutions will be graded both for correctness and quality.

Optional extra space for problem 50.

51. () Imagine a clockshop full of dozens of mechanical clocks. Further imagine the the clockmaker is... particular and simply can't stand to hear "tick... tick" or "tock... tock". Our clockmaker does not care which clock tocks after each tick or ticks after each tock, but a harmonious "tick... tock" should be the only sound throughout the day.

[10] You are under no obligation to fill all the space available. Some people write larger than others.
[11] You are under no obligation to fill all the space available. Some people write larger than others.

- Assume for the purposes of this problem that there are N clocks running concurrently, each of which runs one of the following function (chosen randomly):

```
void ticker(int self) {
  while(!wounddown()) {
    tick(self);
    /* wait 1/2s */
    tock(self);
  }
}
```

```
void tocker(int self) {
  while(!wounddown()) {
    tock(self);
    /* wait 1/2s */
    tick(self);
  }
}
```

- You may further assume that the clocks start randomly wantiing to
- The clockmaker doesn't want any clock held up longer than necessary. That is, you should attempt to maximize concurrency in your solution (after correctness, of course).

Using semaphores, and the C-like syntax we used in class ("`Semaphore`" to declare a semaphore and "`UP()`" and "`DOWN()`" for the operations), develop a solution to the synchronization problem described above by implementing the following three functions:

`void tick(int id)`	Called when a thread wants to tick.
`void tock(int id)`	Called when a thread wants to tock.
`void init()`	Called at the beginning of time before any clocks are created.

Explain briefly how your solution solves the problem.

Think carefully, *then* write[12]. Solutions will be graded both for correctness and quality.

Optional extra space for problem 51.

52. () Imagine that a pandemic has shut down large parts of the world and requires people to stay at least six feet apart from each other. Further, imagine that this disease has mysteriously caused all toilet paper to disappear requiring residents to (cautiously) venture out to Costco in search of resupply.

In order to enforce distancing rules, Costco only allows N customers in the store at a time, but the cart corral is only big enough to allow one customer in it at a time while maintaining safe distancing. Because of (a) the vast quantities of snacks[13] required and (b) that it makes the problem work, everyone is required to take a cart on the way in and return it on the way out.

Your task: Using semaphores, and the C-like syntax we used in class ("`Semaphore`" to declare a semaphore and "`UP()`" and "`DOWN()`" for the operations), develop a solution to the synchronization problem described above by implementing the following three functions:

`enter_store()`	called by a customer who wants to enter the store from the parking lot.
`exit_store()`	called by a customer who wants to exit the store and go outside.
`init()`	called at the beginning of the day before any customers arrive.

The rules:

- All customers are modeled as independent processes. There is no limit on the number of customers.
- No more than N customers may be in the store at a time.
- All customers are required to take a cart from the corral on the way in and to return it on the way out.
- No more than one customer may be in the cart corral.
- Describe actions with comments. E.g.: *"/* enter cart corral */", "/* enter store */", "/* take TP */", etc.*

[12] You are under no obligation to fill all the space available. Some people write larger than others.
[13] Consider this comic.

Explain briefly how your solution solves the problem.

Think carefully, *then* write[14]. Solutions will be graded both for correctness and quality.

Optional extra space for problem 52.

53. () One of the alleged attractions of having children[15] is hearing the pitter patter of little feet. This is all well and good while there is only one child, but once there's more than one it becomes increasingly likely that one might hear "pitter pitter" followed by "patter patter." This cannot be tolerated.

    Imagine a nursery full of dozens of children. Imagine, too, a nanny, Scary Poppins, who simply will not abide disorder. Your job is to ensire that Poppins only hears a harmonious "pitter patter."

    - Assume for the purposes of this problem that there are N children running concurrently[16], each of which executes one of the following functions (chosen at random, but there will be at least one of each):

```
void even(int self) {
  while(!wounddown()) {
    pitter(self);
    /* wait 1/2s */
    patter(self);
  }
}
```

```
void odd(int self) {
  while(!wounddown()) {
    patter(self);
    /* wait 1/2s */
    pitter(self);
  }
}
```

    - You should attempt to maximize concurrency in your solution (after correctness, of course).

    Using semaphores, and the C-like syntax we used in class ("`Semaphore`" to declare a semaphore and "`UP()`" and "`DOWN()`" for the operations), develop a solution to the synchronization problem described above by implementing the following three functions:

`void pitter(int id)`	Called when a child wants to pitter.
`void patter(int id)`	Called when a child wants to patter.
`void init()`	Called at the beginning of time before any children are created.

    Explain briefly how your solution solves the problem.

    Think carefully, *then* write[17]. Solutions will be graded both for correctness and quality.

    Optional extra space for problem 53.

54. () Assume you have hacked the kernel and are using a version of MINIX with a 1 second(!) quantum. Suppose there are five user-level jobs, labeled A through E, that require the following amounts of time to run (if they could get the whole CPU to themselves).

Job	Time
A	2 sec.
B	3 sec.
C	4 sec.
D	5 sec.
E	2 sec.

    Supposing that these five jobs become runnable at very close to the same time and are initially added to the run queue in alphabetical order, answer the following questions.

    Recall that within a priority class, MINIX uses round-robin scheduling. You may assume that these are the only five jobs in the system and that the overhead required for context switches is negligible.

[14] You are under no obligation to fill all the space available. Some people write larger than others.
[15] "Having children must be confusing and difficult." — Thing One, 2/8/2026
[16] Don't they always, no matter how often you've told them not to
[17] You are under no obligation to fill all the space available. Some people write larger than others.

(a) After how many seconds will each job terminate? Briefly explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

(b) Now suppose that job D issues an I/O request after each 1.4s it is allowed to run. This I/O request takes 200ms (0.2s) to complete and causes D to block. Now how long will it take for each job to complete? Again, explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

55. () Assume you have hacked the kernel and are using a version of MINIX with a 1 second(!) quantum. Suppose there are five user-level jobs, labeled A through E, that require the following amounts of time to run (given the whole CPU).

Job	Time
A	5 sec.
B	3 sec.
C	4 sec.
D	2 sec.
E	1 sec.

Supposing that these five jobs become runnable at very close to the same time and are initially added to the run queue in alphabetical order, answer the following questions.

Recall that within a priority class, MINIX uses round-robin scheduling. You may assume that these are the only five jobs in the system and that the overhead required for context switches is negligible.

If a blocked job becomes runnable at the same time another process's quantum is up, the now unblocked job is added to the run queue ahead of the other job.

(a) After how many seconds will each job terminate? Briefly explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

(b) Now suppose that job C issues an I/O request after each 1.1s it is allowed to run. This I/O request takes 500ms (0.5s) to complete and causes C to block. Now how long will it take for each job to complete? Again, explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

56. () Assume you have hacked the kernel and are using a version of MINIX with a 1 second(!) quantum. Suppose there are five user-level jobs, labeled A through E, that require the following amounts of time to run (given the whole CPU).

Job	Time
A	4 sec.
B	3 sec.
C	2 sec.
D	1 sec.
E	5 sec.

Supposing that these five jobs become runnable at very close to the same time and are initially added to the run queue in alphabetical order, answer the following questions.

Recall that within a priority class, MINIX uses round-robin scheduling. You may assume that these are the only five jobs in the system and that the overhead required for context switches is negligible.

If a blocked job becomes runnable at the same time another process's quantum is up, the now unblocked job is added to the run queue ahead of the other job.

(a) After how many seconds will each job terminate? Briefly explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

(b) Now suppose that job A issues an I/O request after each 1.3s it is allowed to run. This I/O request takes 700ms (0.7s) to complete and causes A to block. Now how long will it take for each job to complete? Again, explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

57. () Assume you have hacked the kernel and are using a version of MINIX with a 1 second(!) quantum. Suppose there are five user-level jobs, labeled A through E, that require the following amounts of time to run (given the whole CPU).

Job	Time
A	3 sec.
B	1 sec.
C	2 sec.
D	1 sec.
E	3 sec.

Supposing that these five jobs become runnable at very close to the same time and are initially added to the run queue in alphabetical order, answer the following questions.

Recall that within a priority class, MINIX uses round-robin scheduling. You may assume that these are the only five jobs in the system and that the overhead required for context switches is negligible.

If a blocked job becomes runnable at the same time another process's quantum is up, the (now unblocked) job is added to the run queue ahead of the other job.

(a) After how many seconds will each job terminate? Briefly explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

(b) Now suppose that job C issues an I/O request after each 0.5s it is allowed to run. This I/O request takes 1.6s to complete and causes C to block. Now how long will it take for each job to complete? Again, explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

58. () Consider an operating system that provides **simple round-robin**[18] preemptive scheduling with a 1 second(!) quantum. Suppose there are five user-level jobs, labeled A through E, that require the following amounts of time to run (if they could get the whole CPU to themselves).

[18] That is, each process takes its turn then goes to the back of the line.

Given that these five jobs become runnable at very close to the same time and are initially added to the run queue in alphabetical order, answer the following questions.
Recall that within a priority class, MINIX uses round-robin scheduling. You may assume that these are the only five jobs in the system and that the overhead required for context switches is negligible.
Tiebreaker: If a blocked job becomes runnable at the same time another process's quantum is up, the newly unblocked job is added to the tail of the run queue ahead of the job whose quantum is ending.

Job	Time
A	1 sec.
B	1 sec.
C	2 sec.
D	3 sec.
E	5 sec.

(a) After how many seconds will each job terminate? Briefly explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

(b) Now suppose that job D issues an I/O request after each 1.2s it is allowed to run. This I/O request takes 1.5s to complete and causes D to block. Now how long will it take for each job to complete? Again, explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

59. () Consider an operating system that provides simple round-robin preemptive scheduling with a 1 second(!) quantum. Suppose there are five user-level jobs, labeled A through E, that require the following amounts of time to run (if they could get the whole CPU to themselves).

Given that these five jobs become runnable at very close to the same time and are initially added to the run queue in alphabetical order, answer the following questions.
You may assume that these are the only five jobs in the system and that the overhead required for context switches is negligible.
Tiebreaker: If a blocked job becomes runnable at the same time another process's quantum is up, the newly unblocked job is added to the run queue ahead of the job whose quantum is ending.

Job	Time
A	1.0 sec.
B	1.5 sec.
C	2.0 sec.
D	2.5 sec.
E	3.0 sec.

(a) After how many seconds will each job terminate? Briefly explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

(b) Now suppose that job B issues an I/O request after each 0.4s it is allowed to run. This I/O request takes 1.3s to complete and causes B to block. Now how long will it take for each job to complete? Again, explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

60. () Consider an operating system that provides simple round-robin preemptive scheduling with a 1 second(!) quantum. Suppose there are five user-level jobs, labeled A through E, that require the following amounts of time to run (if they could get the whole CPU to themselves).

Given that these five jobs become runnable at very close to the same time and are initially added to the run queue in alphabetical order, answer the following questions.
You may assume that these are the only five jobs in the system and that the overhead required for context switches is negligible.
Tiebreaker: If a blockd job becomes runnable at the same time another process's quantum is up, the newly unblocked job is added to the run queue ahead of the job whose quantum is ending.

Job	Time
A	3.0 sec.
B	1.0 sec.
C	4.0 sec.
D	1.0 sec.
E	5.0 sec.

(a) After how many seconds will each job terminate? Briefly explain your reasoning. (A scheduling diagram is sufficient explaination.)

Job	Finish Time
A	
B	
C	
D	
E	

(b) Now suppose that job C issues an I/O request after each 1.5s it is allowed to run. This I/O request takes 1.5s to complete and causes C to block. Now how long will it take for each job to complete? Again, explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

61. () Consider an operating system that provides **simple round-robin**[19] preemptive scheduling with a 1 second(!) quantum. Suppose there are five user-level jobs, labeled A through E, that require the following amounts of time to run (if they could get the whole CPU to themselves).

Job	Time
A	1.5 sec.
B	2.0 sec.
C	2.5 sec.
D	2.0 sec.
E	1.5 sec.

Given that these five jobs become runnable at very close to the same time and are initially added to the run queue in alphabetical order, answer the following questions.
You may assume that these are the only five jobs in the system and that the overhead required for context switches is negligible.
Tiebreaker: If a blocked job becomes runnable at the same time another process's quantum is up, the newly unblocked job is added to the run queue ahead of the job whose quantum is ending.
If ASCII art isn't your thing, submit a picture separately to show work.

(a) After how many seconds will each job terminate? Briefly explain your reasoning. (A scheduling diagram is sufficient explaination.)

Job	Finish Time
A	
B	
C	
D	
E	

[19] That is, each process takes its turn then goes to the back of the line.

(b) Now suppose that job C issues an I/O request after each 0.5s it is allowed to run. This I/O request takes 1.0s to complete and causes C to block. Now how long will it take for each job to complete? Again, explain your reasoning.

Job	**Finish Time**
A	
B	
C	
D	
E	

62. () Consider an operating system that provides **simple round-robin**[20] preemptive scheduling with a 1 second(!) quantum. Suppose there are five user-level jobs, labeled A through E, that require the following amounts of time to run (if they could get the whole CPU to themselves).

[20] That is, each process takes its turn then goes to the back of the line.

Given that these five jobs become runnable at very close to the same time and are initially added to the run queue in alphabetical order, answer the following questions.
You may assume that these are the only five jobs in the system and that the overhead required for context switches is negligible.
Tiebreaker: If a blocked job becomes runnable at the same time another process's quantum is up, the newly unblocked job is added to the run queue ahead of the job whose quantum is ending.

Job	Time
A	3.0 sec.
B	2.5 sec.
C	1.0 sec.
D	3.0 sec.
E	3.0 sec.

(a) After how many seconds will each job terminate? Briefly explain your reasoning. (A scheduling diagram is sufficient explanation.)

Job	Finish Time
A	
B	
C	
D	
E	

(b) Now suppose that job C issues an I/O request after each 0.5s it is allowed to run. This I/O request takes 0.5s to complete and causes C to block. Now how long will it take for each job to complete? Again, explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

This was supposed to have been job B, not C, which would've led to an interesting schedule.

63. () Consider an operating system that provides **simple round-robin**[21] preemptive scheduling with a 1 second(!) quantum. Suppose there are five user-level jobs, labeled A through E, that require the following amounts of time to run (if they could get the whole CPU to themselves).

Given that these five jobs become runnable at very close to the same time and are initially added to the run queue in alphabetical order, answer the following questions.
You may assume that these are the only five jobs in the system and that the overhead required for context switches is negligible.
Tiebreaker: If a blocked job becomes runnable at the same time another process's quantum is up, the newly unblocked job is added to the run queue ahead of the job whose quantum is ending.

Job	Time
A	3.0 sec.
B	2.5 sec.
C	1.0 sec.
D	3.0 sec.
E	3.0 sec.

[21] That is, each process takes its turn then goes to the back of the line.

(a) After how many seconds will each job terminate? Briefly explain your reasoning. (A scheduling diagram is sufficient explanation.)

Job	**Finish Time**
A	
B	
C	
D	
E	

(b) Now suppose that job B issues an I/O request after each 0.5s it is allowed to run. This I/O request takes 0.5s to complete and causes B to block. Now how long will it take for each job to complete? Again, explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

64. () Consider an operating system that provides **simple round-robin**[22] preemptive scheduling with a 1 second(!) quantum. Suppose there are five user-level jobs, labeled A through E, that require the following amounts of time to run (if they could get the whole CPU to themselves).

Job	Time
A	3.0 sec.
B	2.0 sec.
C	1.0 sec.
D	3.0 sec.
E	2.0 sec.

Given that these five jobs become runnable at very close to the same time and are initially added to the run queue in alphabetical order, answer the following questions.
You may assume that these are the only five jobs in the system and that the overhead required for context switches is negligible.
Tiebreaker: If a blocked job becomes runnable at the same time another process's quantum is up, the newly unblocked job is added to the tail of the run queue ahead of the job whose quantum is ending.

(a) After how many seconds will each job terminate? Briefly explain your reasoning. (A scheduling diagram is sufficient explanation.)

Job	Finish Time
A	
B	
C	
D	
E	

[22] That is, each process takes its turn then goes to the back of the line.

(b) Now suppose that job B issues an I/O request after each 0.5s it is allowed to run. This I/O request takes 1.5s to complete and causes B to block. Now how long will it take for each job to complete? Again, explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

65. () Consider an operating system that provides **simple round-robin**[23] preemptive scheduling with a 1 second(!) quantum. Suppose there are five user-level jobs, labeled A through E, that require the following amounts of time to run (if they could get the whole CPU to themselves).

Job	Time
A	5.0 sec.
B	4.0 sec.
C	3.0 sec.
D	2.0 sec.
E	1.0 sec.

Given that these five jobs become runnable at very close to the same time and are initially added to the run queue in alphabetical order, answer the following questions.
You may assume that these are the only five jobs in the system and that the overhead required for context switches is negligible.
Tiebreaker: If a blocked job becomes runnable at the same time another process's quantum is up, the newly unblocked job is added to the tail of the run queue ahead of the job whose quantum is ending.

(a) After how many seconds will each job terminate? Briefly explain your reasoning. (A scheduling diagram is sufficient explanation.)

Job	Finish Time
A	
B	
C	
D	
E	

[23] That is, each process takes its turn then goes to the back of the line.

(b) Now suppose that job B issues an I/O request after each 1.5s it is allowed to run. This I/O request takes 1.5s to complete and causes B to block. Now how long will it take for each job to complete? Again, explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

qq

66. () Consider an operating system that provides **simple round-robin**[24] preemptive scheduling with a 1 second(!) quantum. Suppose there are five user-level jobs, labeled A through E, that require the following amounts of time to run (if they could get the whole CPU to themselves).

Given that these five jobs become runnable at very close to the same time and are initially added to the run queue in alphabetical order, answer the following questions.
You may assume that these are the only five jobs in the system and that the overhead required for context switches is negligible.
Tiebreaker: If a blocked job becomes runnable at the same time another process's quantum is up, the newly unblocked job is added to the tail of the run queue ahead of the job whose quantum is ending.

Job	Time
A	1.0 sec.
B	2.0 sec.
C	3.0 sec.
D	4.0 sec.
E	5.0 sec.

(a) After how many seconds will each job terminate? Briefly explain your reasoning. (A scheduling diagram is sufficient explanation.)

Job	Finish Time
A	
B	
C	
D	
E	

[24] That is, each process takes its turn then goes to the back of the line.

(b) Now suppose that job C issues an I/O request after each 0.5s it is allowed to run. This I/O request takes 1.5s to complete and causes C to block. Now how long will it take for each job to complete? Again, explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

67. () Consider an operating system that provides **simple round-robin**[25] preemptive scheduling with a 1 second(!) quantum. Suppose there are five user-level jobs, labeled A through E, that require the following amounts of time to run (if they could get the whole CPU to themselves).

Job	Time
A	1.0 sec.
B	4.0 sec.
C	3.0 sec.
D	2.0 sec.
E	5.0 sec.

Given that these five jobs become runnable at very close to the same time and are initially added to the run queue in alphabetical order, answer the following questions.
You may assume that these are the only five jobs in the system and that the overhead required for context switches is negligible.
Tiebreaker: If a blocked job becomes runnable at the same time another process's quantum is up, the newly unblocked job is added to the tail of the run queue ahead of the job whose quantum is ending.

(a) After how many seconds will each job terminate? Briefly explain your reasoning. (A scheduling diagram is sufficient explanation.)

Job	Finish Time
A	
B	
C	
D	
E	

[25] That is, each process takes its turn then goes to the back of the line.

(b) Now suppose that job C issues an I/O request after each 0.5s it is allowed to run. This I/O request takes 1.5s to complete and causes C to block. Now how long will it take for each job to complete? Again, explain your reasoning.

Job	Finish Time
A	
B	
C	
D	
E	

68. () Consider an operating system that provides **simple round-robin**[26] preemptive scheduling with a 1 second(!) quantum. Suppose there are five user-level jobs, labeled A through E, that require the following amounts of time to run (if they could get the whole CPU to themselves).

Job	Time
A	1.0 sec.
B	4.0 sec.
C	2.5 sec.
D	2.0 sec.
E	5.0 sec.

Given that these five jobs become runnable at very close to the same time and are initially added to the run queue in alphabetical order, answer the following questions.
You may assume that these are the only five jobs in the system and that the overhead required for context switches is negligible.
Tiebreaker: If a blocked job becomes runnable at the same time another process's quantum is up, the newly unblocked job is added to the tail of the run queue ahead of the job whose quantum is ending.

(a) After how many seconds will each job terminate? Briefly explain your reasoning. (A scheduling diagram is sufficient explanation.)

Job	Finish Time
A	
B	
C	
D	
E	

[26] That is, each process takes its turn then goes to the back of the line.

(b) Now suppose that job C issues an I/O request after each 1.0s it is allowed to run. This I/O request takes 1.5s to complete and causes C to block. Now how long will it take for each job to complete? Again, explain your reasoning.

Job	**Finish Time**
A	
B	
C	
D	
E	